UNIVERSIDADE FEDERAL DO AMAZONAS
INSTITUTO DE CIÊNCIAS EXATAS E TECNOLOGIA
PROGRAMA DE PÓS-GRADUAÇÃO EM
CIÊNCIAS, TECNOLOGIA E SAÚDE

Inteligência Artificial Embarcada

Segundo trabalho — Grupo 03

Camila Amorim dos Santos
Gabriel dos Santos Tavares
Gabriel Iuji Oliveira Taguchi
Jéssica Corsino Ribeiro
Nathália Garcez Galindo Brandão

MS-Monitor: arquitetura de monitoramento e classificação embarcada de condições de armazenamento de grãos

Itacoatiara–AM
2026

Camila Amorim dos Santos
Gabriel dos Santos Tavares
Gabriel Iuji Oliveira Taguchi
Jéssica Corsino Ribeiro
Nathália Garcez Galindo Brandão

MS-Monitor: arquitetura de monitoramento e classificação embarcada de condições de armazenamento de grãos

Relatório técnico apresentado como segundo trabalho da disciplina Inteligência Artificial Embarcada, do Programa de Pós-Graduação em Ciências, Tecnologia e Saúde da Universidade Federal do Amazonas, Instituto de Ciências Exatas e Tecnologia, no segundo semestre de 2026.

Docente: Prof. Dr. Carlos Alberto Oliveira de Freitas.

Grupo 03.

Itacoatiara–AM
2026

Sumário

1 Introdução e delimitação do trabalho 1

1.1 Relação com a atividade do Grupo 03 1

1.2 Objetivo e limites de interpretação 1

2 Arquitetura funcional e alimentação 2

2.1 Caminhos de energia e de retorno 4

2.2 Grandezas elétricas e dimensionamento pendente 4

3 Aquisição e preparação dos sinais 5

3.1 Características dos componentes 5

3.2 Periodicidade e janela de vibração 7

3.3 Validação, atributos e tratamento de falhas 7

4 Modelo de IA e implantação no firmware 8

4.1 Treinamento no computador e inferência no dispositivo 8

4.2 Saídas do classificador e seu significado 8

4.3 Execução e independência da rede 10

5 Comunicação, armazenamento e visualização 12

5.1 Contrato de transmissão 12

5.2 Consulta e persistência 13

5.3 Apresentação e limites de comunicação 13

6 Evidências disponíveis e plano de testes 14

6.1 Resultados computacionais do modelo 14

6.2 Verificação de software e da exportação 15

6.3 Plano de testes físicos e de integração 15

7 Conclusões sobre a viabilidade da arquitetura 17

Declaração de apoio de inteligência artificial 18

Referências **19**

1 Introdução e delimitação do trabalho

O acompanhamento das condições de armazenamento de grãos pode apoiar a investigação de alterações no ambiente de conservação. Temperatura e umidade estão entre as variáveis relevantes nesse contexto, mas uma medição ambiental isolada não identifica o agente causador de uma alteração. A Organização Mundial da Saúde descreve a relação entre determinadas condições de armazenamento, crescimento de fungos e ocorrência de micotoxinas, sem equiparar essas três situações (World Health Organization, 2023). No presente trabalho, a investigação é restrita à arquitetura de um protótipo de monitoramento e à execução local de um classificador demonstrativo.

O MS-Monitor integra uma Heltec WiFi LoRa 32 V2, um DHT22, um termopar tipo K com conversor MAX6675 e um módulo de vibração SW-420. O firmware adquire as medições, prepara quatro atributos térmicos e higrométricos e executa um Random Forest na ESP32. As medições e a classificação são transmitidas a uma API local para registro em CSV e apresentação em uma interface web. O sensor de vibração produz telemetria independente; seus dados não são entradas do modelo atual.

1.1 Relação com a atividade do Grupo 03

A especificação original do Grupo G3 solicita a representação dos módulos de aquisição, processamento, IA, comunicação e visualização, além de um fluxograma, características técnicas, exemplo de transmissão e plano de testes (FREITAS, 2026). Naquele enunciado, a IA está associada ao estado operacional da ventilação. A proposta aqui documentada foi redirecionada pela equipe ao monitoramento das condições de armazenamento.

Este relatório utiliza os entregáveis arquiteturais da atividade como referência e explicita essa mudança de objeto. Não apresenta um classificador de ventilação, nem inclui motor, relé ou atuação automática como partes implementadas. Consequentemente, não atribui ao SW-420 a capacidade de determinar, por uma contagem isolada, se um ventilador está ligado.

1.2 Objetivo e limites de interpretação

O objetivo é descrever e avaliar a coerência técnica do caminho dos dados, desde os sensores até a visualização, distinguindo implementação em software, verificação computacional e validação experimental. Especificamente, o trabalho apresenta as ligações e a alimentação, o pré-processamento efetivamente implementado, a exportação do modelo para C++, o contrato da API e um plano de testes físicos.

A hipótese de pesquisa é investigar se medições térmicas e de umidade podem contribuir para a classificação de condições associadas à deterioração. **O protótipo não demonstrou detecção de fungos, identificação de espécies, presença de micotoxinas ou segurança sanitária dos grãos.** Temperatura na ponta da sonda não caracteriza toda a massa armazenada; umidade relativa do ar também não equivale à umidade do grão ou à atividade de água. A confirmação dessas condições exige referências e procedimentos experimentais adicionais.

As descrições técnicas foram confrontadas com o código do projeto, a documentação do modelo e os resultados computacionais disponíveis na revisão `07b1400` (Grupo 03, 2026). Não se pressupõe que a existência do firmware compilável comprove, por si só, calibração ou desempenho em bancada.

2 Arquitetura funcional e alimentação

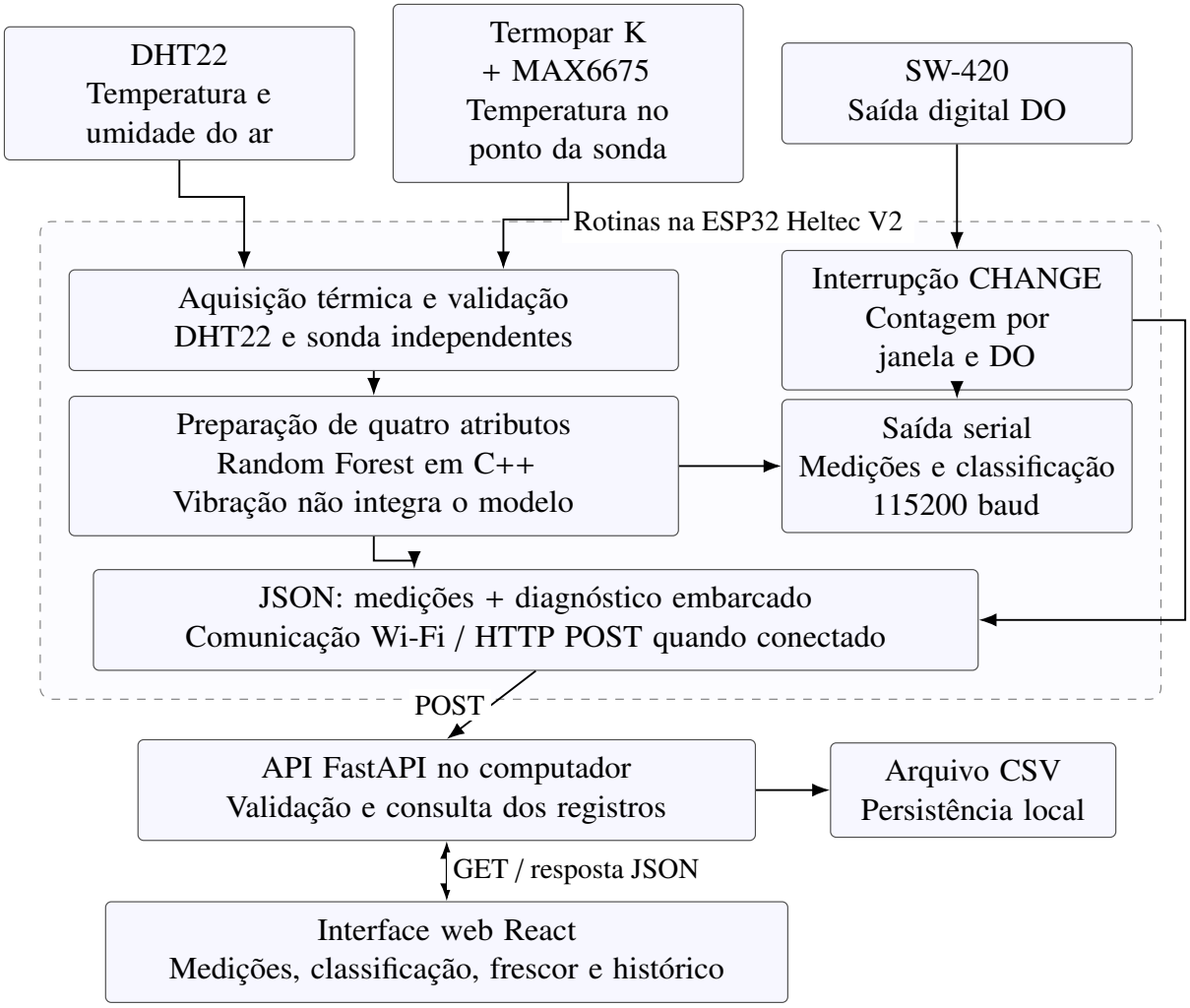
A Figura 1 diferencia as rotinas térmicas utilizadas na classificação da contagem de transições do SW-420. Os blocos internos representam funções executadas na mesma ESP32, e não microcontroladores distintos. O treinamento ocorre previamente no computador; o dispositivo executa somente a inferência do modelo exportado.

Tabela 1 – Responsabilidade dos módulos e limites da implementação

Módulo	Função no protótipo
Aquisição	Ler DHT22 e MAX6675; capturar mudanças do sinal digital de vibração por interrupção.
Pré-processamento	Validar as entradas, preservar indicadores de falha e calcular a diferença entre temperaturas.
IA embarcada	Executar o Random Forest exportado; produzir classe, índice, confiança e duração da chamada.
Interface local	Registrar medições e resultado na serial a 115200 baud, inclusive sem Wi-Fi.
Comunicação	Realizar uma tentativa de POST por leitura quando houver Wi-Fi; tratar o código de retorno.
API e armazenamento	Validar o contrato, registrar cada POST aceito em CSV e disponibilizar consultas.
Visualização	Exibir medições, origem da classificação, frescor dos dados e histórico no navegador.
Atuação	Não implementada; não há comando de ventilador no firmware atual.

Fonte: elaboração própria a partir do código do projeto.

Figura 1 – Arquitetura implementada: aquisição, inferência local e serviços externos

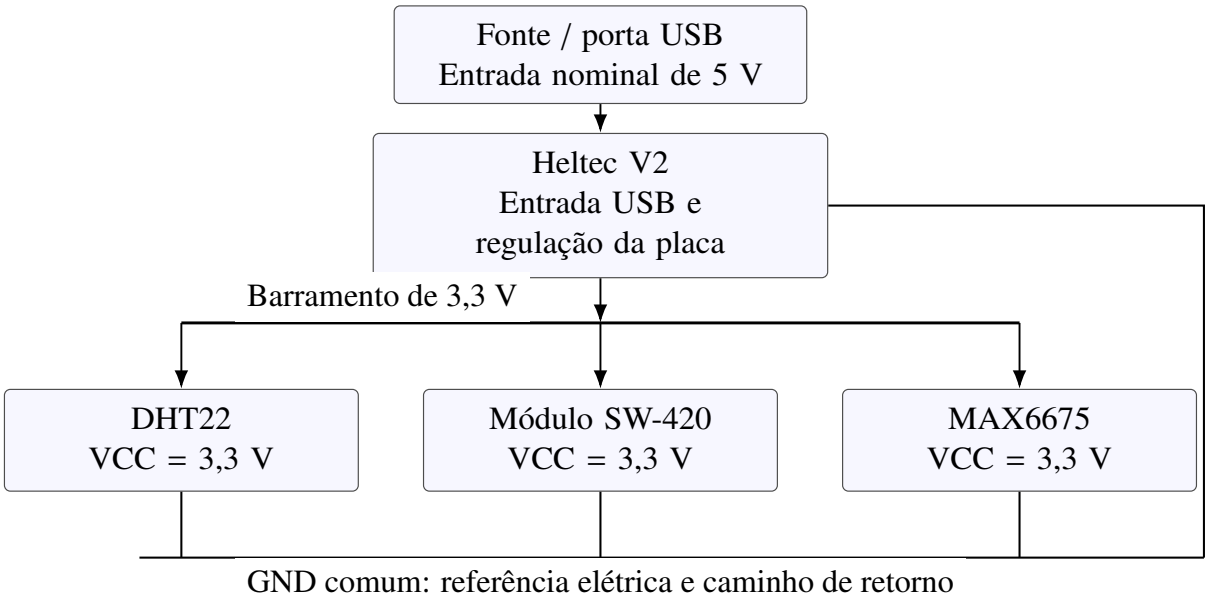


Fonte: elaboração própria a partir do firmware, da API e do frontend do MS-Monitor.

2.1 Caminhos de energia e de retorno

A alimentação adotada é USB, com entrada nominal de 5 V na placa e distribuição de 3,3 V aos módulos sensores. O terra é comum, conforme a Figura 2. As linhas de alimentação e GND não representam o fluxo lógico de dados. O termopar é passivo e se conecta às entradas próprias do MAX6675; não recebe alimentação de 3,3 V na ponta sensora.

Figura 2 – Distribuição de alimentação adotada e referência de terra comum



Fonte: elaboração própria a partir das ligações documentadas; diagrama funcional, não esquema completo da placa.

Os sinais que chegam aos GPIOs devem ser compatíveis com a lógica de 3,3 V da ESP32. A placa utilizada possui recursos adicionais de comunicação e visualização, mas LoRa, Bluetooth e OLED não são utilizados pelo firmware analisado (Heltec Automation, s.d.; Espressif Systems, s.d.). O GPIO 21, associado ao controle de Vext nessa montagem, não é destinado aos sensores.

2.2 Grandezas elétricas e dimensionamento pendente

O dimensionamento final requer a identificação da fonte USB, a corrente dos módulos efetivamente montados e os picos durante a comunicação. Esses valores não foram determinados pelos testes de software. Assim, não se apresenta uma soma de correntes hipotéticas como consumo medido do protótipo. A potência de entrada deve ser obtida no mesmo ponto de medição:

$$P_{\mathrm{USB}} = V_{\mathrm{USB}} I_{\mathrm{USB}}. \tag{1}$$

Tabela 2 – Mapa de conexões utilizado no firmware

Sinal	GPIO	Interface e direção em relação à ESP32
DHT22 — DAT	17	Protocolo digital temporizado, bidirecional.
SW-420 — DO	22	Entrada digital; interrupção em `CHANGE`.
MAX6675 — SCK	13	Saída de relógio gerada por software.
MAX6675 — CS	23	Saída de seleção do conversor.
MAX6675 — SO	39	Entrada de dados; GPIO de entrada.
VCC dos módulos	—	Barramento de 3,3 V.
GND dos módulos	—	Retorno comum ao GND da placa.

Fonte: elaboração própria a partir das constantes e rotinas do firmware.

Já a potência entregue apenas aos sensores no barramento de 3,3 V é

$$P_{\text{sensores}} = 3{,}3\,(I_{\text{DHT22}} + I_{\text{MAX6675}} + I_{\text{SW420}})\,, \tag{2}$$

com correntes em ampères e potência em watts. Essa expressão não inclui todo o consumo da placa, nem as perdas de regulação.

Como exemplo exclusivamente didático, para um resistor ideal de 10 kΩ submetido a 3,3 V, a Lei de Ohm fornece

$$I = \frac{V}{R} = \frac{3{,}3}{10\,000} = 0{,}00033\ \text{A} = 0{,}33\ \text{mA}. \tag{3}$$

Esse resistor não é apresentado como componente identificado da montagem. O exemplo apenas ilustra a relação entre tensão, resistência e corrente. O plano de testes da Seção 6 prevê a medição da entrada USB e do barramento de 3,3 V antes de concluir sobre a suficiência da fonte.

3 Aquisição e preparação dos sinais

3.1 Características dos componentes

A Tabela 3 reúne informações de referência e decisões da montagem. Especificações de catálogo não equivalem a erros determinados no protótipo: posicionamento, fiação, alimentação e características do módulo podem afetar as leituras. Também é necessário distinguir resolução digital, erro especificado e incerteza de uma comparação experimental.

No MAX6675, a resolução de 0,25 °C não significa que o conjunto termopar–conversor tenha esse erro máximo. O documento do fabricante estabelece, por exemplo, erro do conversor de até 8 LSB na faixa de 0 a 700 °C, com alimentação de 3,3 V e temperatura ambiente especificada, além de tratar separadamente a compensação da junção fria (Analog Devices,

Tabela 3 – Características técnicas relevantes ao protótipo

Componente	Características de referência	Uso e limitação no projeto
Heltec WiFi LoRa 32 V2	ESP32 clássico, dois núcleos LX6 até 240 MHz, 520 KiB de SRAM no chip e 8 MB de flash na placa; Wi-Fi de 2,4 GHz.	Aquisição e inferência local. A memória disponível à aplicação depende da partição e dos recursos do firmware.
DHT22 / AM2302	Temperatura de −40 a 80 °C; UR de 0 a 100%; alimentação de referência de 3 a 5 V; novas leituras em intervalos de pelo menos 2 s.	Alimentado em 3,3 V. A referência consultada informa erro de ±0,5 °C e de 2 a 5 pontos percentuais de UR; não aferidos no protótipo.
Termopar tipo K	Sensor termoelétrico passivo; a conversão depende do MAX6675 e da compensação da junção fria.	Mede o ponto em que a ponta está instalada. Não caracteriza, sozinho, o campo térmico de toda a massa de grãos.
MAX6675	Conversor de 12 bits, resolução de 0,25 °C, alimentação de 3,0 a 5,5 V e conversão de até 220 ms.	Alimentado em 3,3 V; leitura serial por software. O código aceita 0 a 1024 °C; o maior código de 12 bits equivale a 1023,75 °C.
Módulo SW-420	Saída digital de um módulo de detecção de vibração; o ajuste e a polaridade dependem da implementação utilizada.	Alimentado em 3,3 V. Registra DO e transições; não fornece aceleração, intensidade ou frequência mecânica calibradas.

Fonte: especificações de referência de Heltec, Espressif, Adafruit e Analog Devices; limites de uso extraídos do firmware (Heltec Automation, s.d.; Espressif Systems, s.d.; Adafruit Industries, s.d.; Analog Devices, 2021).

2021). Essas condições não constituem calibração da sonda montada. Para o SW-420 não foi identificado um certificado ou uma especificação metrológica do módulo efetivamente utilizado; por isso, não se atribuem unidades físicas à contagem digital.

3.2 Periodicidade e janela de vibração

O firmware estabelece um intervalo nominal de 2500 ms. Na ausência de atrasos adicionais, a taxa de ciclos é

$$f_{\text{nominal}} = \frac{1}{2{,}5\ \text{s}} = 0{,}4\ \text{Hz}, \tag{4}$$

equivalente a aproximadamente 24 ciclos por minuto. Essa é a taxa nominal de aquisição e envio, não uma frequência de amostragem de uma forma de onda de vibração. A duração real é transmitida em `window_ms`, pois atrasos de rede ou de execução podem prolongar a janela.

As transições do SW-420 são capturadas por uma interrupção em `CHANGE`; o contador é copiado e reiniciado em seção crítica. O fechamento da janela antecede a leitura dos sensores térmicos, portanto o registro não deve ser descrito como uma aquisição simultânea de todas as grandezas. A rotina de interrupção não faz HTTP, leitura do DHT22 ou impressão serial. Como a biblioteca do DHT pode bloquear interrupções brevemente, a contagem continua sendo uma medida demonstrativa.

A janela inicial também fornece tempo superior aos 220 ms de conversão do MAX6675. A leitura ocorre uma vez por ciclo, utilizando o construtor na ordem SCK, CS e SO. Não se pressupõe o uso de I^2C: o DHT22 usa seu protocolo temporizado, e o MAX6675 utiliza uma interface serial compatível com SPI implementada por software.

3.3 Validação, atributos e tratamento de falhas

O DHT22 é considerado válido quando temperatura e UR são finitas e estão nos limites aceitos pelo firmware. Se falhar, ambos os valores são enviados como `null`, com `dht_ok=false`. A sonda é validada de forma independente; termopar aberto pode resultar em `NAN` na biblioteca, tratado como temperatura `null` e `probe_ok=false`. Temperatura igual a zero é uma entrada possível e não identifica falha.

Somente com os dois sensores térmicos válidos é preparado o vetor

$$\mathbf{x} = [T_{\text{ar}},\ \text{UR}_{\text{ar}},\ T_{\text{sonda}},\ \Delta T]\,, \qquad \Delta T = T_{\text{sonda}} - T_{\text{ar}}. \tag{5}$$

O pré-processamento reproduz operações em `float32`. A versão embarcada não calcula médias móveis, espectro, tendências ou dias de armazenamento como entradas da IA. Também não implementa filtragem digital de vibração. Essas operações não devem ser atribuídas ao protótipo

apenas por aparecerem em propostas de evolução.

Uma falha térmica torna a classificação indisponível, sem reaproveitar uma classe anterior como se fosse atual. A telemetria de vibração continua sendo enviada. De modo recíproco, uma leitura de vibração igual a zero não impede a inferência térmica.

4 Modelo de IA e implantação no firmware

4.1 Treinamento no computador e inferência no dispositivo

Random Forest combina diferentes árvores treinadas com aleatorização de amostras e atributos (BREIMAN, 2001). No projeto, o treinamento foi implementado com scikit-learn (PEDREGOSA et al., 2011). A configuração compacta tem 32 árvores, profundidade máxima 6, mínimo de cinco amostras por folha, balanceamento `balanced_subsample` e semente 42. Esses parâmetros e o relatório por classe constam do artefato técnico <`docs/embedded-ai-metrics.json`>.

A Figura 3 diferencia treinamento, avaliação e exportação. O script <`ai/train_embedded.py`> gera o cabeçalho <`firmware/ms_monitor/risk_model.h`>. Os parâmetros aprendidos são representados por tabelas constantes de nós e raízes; não são limiares manuais definidos para inferir presença de fungos. O arquivo `.joblib` é mantido no computador para reprodução e comparação e não é carregado pela ESP32 nem pela API atual.

Os dados locais são descritos na documentação como sintéticos: 43 200 registros distribuídos em 20 lotes. Não há, no material examinado, comprovação de que os rótulos derivem de análises microbiológicas dos grãos do protótipo. A identificação do conjunto é preservada por seu hash SHA-256 no relatório de treinamento.

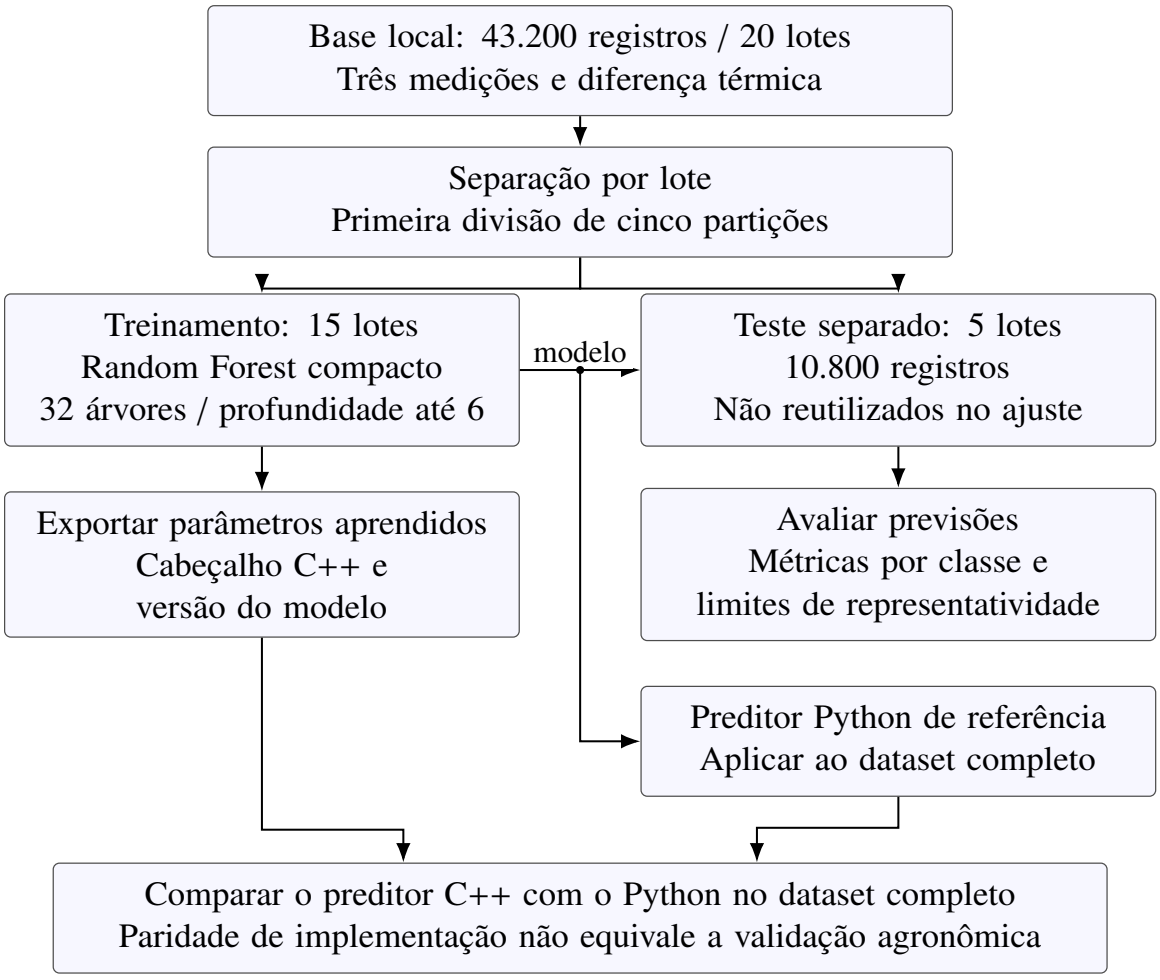
O código usa a **primeira divisão** produzida por `StratifiedGroupKFold` com cinco partições, mantendo lotes inteiros separados. O experimento utiliza 32 400 registros de 15 lotes no treinamento e 10 800 registros de cinco lotes no teste. Não se trata de uma média de desempenho de cinco rodadas de validação cruzada, nem de uma validação temporal independente.

4.2 Saídas do classificador e seu significado

A execução retorna um dos rótulos `low`, `moderate`, `high` ou `critical`. Neste relatório, eles são interpretados como níveis nominais do conjunto de treinamento, respectivamente baixo, moderado, alto e crítico. Esses nomes não estabelecem critérios agronômicos validados nem recomendações automáticas de intervenção.

Para cada classe c, o preditor calcula a média das probabilidades das folhas atingidas

Figura 3 – Preparação do modelo embarcado e verificação da exportação



Fonte: elaboração própria a partir dos scripts de treinamento e verificação.

pelas $M = 32$ árvores:

$$p_c(\mathbf{x}) = \frac{1}{M} \sum_{m=1}^{M} p_{m,c}(\mathbf{x}), \qquad \widehat{c} = \arg\max_c p_c(\mathbf{x}). \tag{6}$$

A confiança reportada é $\max_c p_c(\mathbf{x})$; não corresponde a uma medida experimental de certeza sobre a condição sanitária do lote. O índice apresentado em `risk_score` segue a convenção

$$s = 0\, p_{\text{low}} + 0{,}33\, p_{\text{moderate}} + 0{,}66\, p_{\text{high}} + 1\, p_{\text{critical}}. \tag{7}$$

Os pesos são uma escala de apresentação adotada no código. Mesmo quando expresso como percentual no dashboard, s não é uma probabilidade calibrada de haver fungos ou micotoxinas.

4.3 Execução e independência da rede

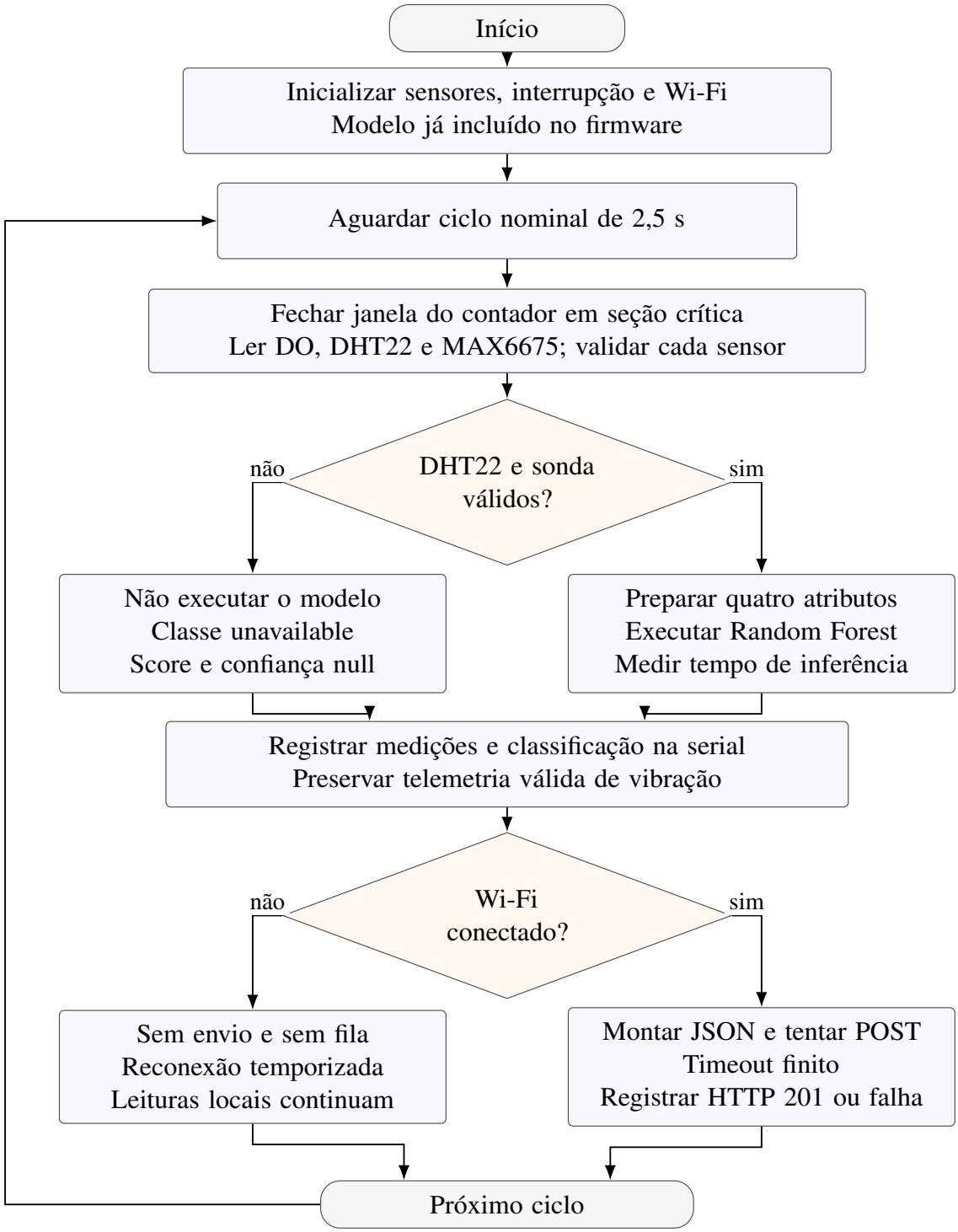
O fluxo da Figura 4 segue a ordem do firmware. O tempo de inferência é medido ao redor da chamada local do preditor com `micros()`, antes de verificar a conectividade. Sem Wi-Fi, o dispositivo continua lendo os sensores, classificando entradas válidas e registrando o resultado na serial. Os dados desse período não são enfileirados para envio posterior.

Quando há rede, ocorre uma tentativa de POST por ciclo. O firmware não testa antecipadamente a disponibilidade da API: interpreta a resposta HTTP após tentar o envio. O código considera 201 como aceitação e registra os demais códigos ou erros. Os limites configurados são 700 ms para conexão e 1000 ms para leitura HTTP; não representam garantia de latência total do ciclo.

O modelo embarcado é identificado como `embedded-rf-6b0d1696547d`. O relatório de compilação registrado no projeto, para core Espressif 3.3.12 e FQBN `<esp32:esp32:heltec_wifi_lora_32_V2>`, informa 1 043 428 bytes de programa, diante de um limite de 3 342 336 bytes, e 48 824 bytes de variáveis globais na região de 327 680 bytes contabilizada pela ferramenta. As tabelas de nós e raízes ocupam 102 496 bytes no compilador nativo de verificação.

Esses números sustentam a análise de compatibilidade da compilação, mas não medem o heap livre em execução, a duração da inferência na placa ou o consumo elétrico. Tampouco se deve confundir a região de memória descrita pelo compilador com a SRAM física total do chip.

Figura 4 – Fluxograma da aquisição e da inferência no firmware



Fonte: elaboração própria a partir de `ms_monitor.ino`; a rotina de interrupção do SW-420 opera em paralelo ao ciclo principal.

5 Comunicação, armazenamento e visualização

5.1 Contrato de transmissão

O envio é realizado pela ESP32 em rede Wi-Fi de 2,4 GHz, por `POST /api/readings`. A API valida tipos, limites e coerência dos indicadores de falha; um registro aceito e persistido retorna HTTP 201. Payloads incompatíveis retornam 422. Falha de gravação do CSV retorna 503, sem substituir a última leitura aceita. O Código 1 utiliza os nomes efetivamente exigidos pelo contrato atual.

Código 1 – Payload ilustrativo compatível com o contrato da API

```
{
  "device_id": "ms-monitor-g3",
  "uptime_ms": 125000,
  "window_ms": 2500,
  "temperature_c": 24.8,
  "humidity_pct": 59.2,
  "dht_ok": true,
  "probe_temperature_c": 25.25,
  "probe_ok": true,
  "vibration_do": 0,
  "vibration_transitions": 12,
  "embedded_diagnosis": {
    "source": "esp32",
    "model_version": "embedded-rf-6b0d1696547d",
    "risk_level": "low",
    "risk_score": 0.12,
    "confidence": 0.8,
    "inference_us": 123
  }
}
```

Todos os valores desse exemplo são **ilustrativos**. Em particular, 123 μs não é uma duração medida na placa. O objeto exemplifica o contrato, e não um ensaio físico. A diferença térmica é calculada internamente para a IA e não constitui um campo próprio no POST atual.

O campo `uptime_ms` representa tempo desde a inicialização da placa, não horário civil. O servidor acrescenta `received_at` em UTC. Esse carimbo corresponde ao recebimento, sujeito ao atraso de comunicação, e não ao instante exato de aquisição. A API não executa outra inferência: apenas valida e prepara o diagnóstico recebido para exibição.

5.2 Consulta e persistência

Tabela 4 – Interfaces HTTP implementadas

Rota	Finalidade
`GET /health`	Saúde da API, independente do estado do dispositivo.
`POST /api/readings`	Recebimento e persistência de uma leitura e de seu diagnóstico embarcado.
`GET /api/readings/latest`	Última leitura, idade, estado de atualização e diagnóstico recebido.
`GET /api/readings?limit=300`	Histórico, em ordem cronológica, com limite de 1 a 3600 registros.
`GET /api/readings/export`	Download do CSV completo até o instante da solicitação.

Fonte: elaboração própria a partir de `backend/app.py`.

O arquivo padrão é <`backend/data/readings.csv`>. Cada POST aceito gera uma linha com as medições, os indicadores de validade e os campos <`ai_source`>, <`ai_model_version`>, <`ai_risk_level`>, <`ai_risk_score`>, <`ai_confidence`> e <`ai_inference_us`>. Campos ausentes ficam em branco no CSV. As colunas de IA registram previsões, e não rótulos independentes de contaminação para treinamento supervisionado.

O histórico em memória é limitado aos últimos 3600 registros e é recuperado do CSV ao iniciar a API. O arquivo completo continua crescendo, portanto a coleta exige acompanhamento do espaço em disco e cópias de segurança. Deve haver uma única instância da API, com um único worker, por arquivo. Após reiniciar, o histórico permanece disponível, mas o estado da última leitura aguarda um novo envio.

O envelope de consulta diferencia `waiting`, antes de um POST aceito pela instância, `online`, para leitura com idade inferior a 10 s, e `stale`, a partir desse intervalo. O frescor é calculado por tempo monotônico no servidor. Um dispositivo com dados recentes ainda pode reportar falha de um sensor; por isso, conectividade e validade das medições são informações distintas.

5.3 Apresentação e limites de comunicação

O dashboard exibe temperatura do ar, temperatura da sonda, UR, transições por janela, última atualização e resultado embarcado com versão e tempo de inferência. A diferença térmica é uma entrada interna do modelo, não um card dedicado na interface examinada. O card de ventilação permanece como *Não avaliado*.

Para os valores ilustrativos do Código 1, a interface apresentaria 24,8 °C de temperatura do ar, 59,2% de UR, 25,25 °C na sonda e 12 transições em uma janela de 2,5 s, com DO igual

a zero. A classe `low` apareceria como *Baixo risco*, com índice de 12%, confiança de 80% e inferência de 123 μs. Essa correspondência exemplifica a formatação dos dados recebidos, sem transformar o exemplo em resultado de ensaio ou o índice em confirmação de contaminação.

A última leitura é consultada aproximadamente a cada segundo; o histórico é sincronizado de forma independente. O modo *Ao vivo* prioriza a coleta atual, enquanto o histórico e o CSV preservam registros anteriores. Uma pausa de recepção não é preenchida com valores artificiais. Na indisponibilidade da API, os valores anteriores continuam visíveis com indicação de desatualização.

A API local usa HTTP e não implementa autenticação de dispositivos. CORS restringe origens de navegador, mas não autentica POSTs enviados pela ESP32. A arquitetura não deve ser apresentada como solução de segurança para acesso público à internet. Credenciais de Wi-Fi ficam em um `config.h` local e não integram o repositório ou este relatório.

6 Evidências disponíveis e plano de testes

6.1 Resultados computacionais do modelo

As métricas da Tabela 5 são provenientes da avaliação registrada para o modelo compacto, nos lotes de teste descritos na Seção 4. Elas avaliam correspondência com rótulos do dataset local, não detecção experimental de fungos no mini-silo.

Tabela 5 – Avaliação do modelo no conjunto de teste

Indicador	Resultado
Registros de teste	10 800
Acurácia geral do Random Forest	98,9907%
Acurácia balanceada	49,9860%
F1 macro, considerando quatro rótulos	24,8732%
Acurácia de prever sempre `low`	99,0185%

Fonte: relatório `embedded-ai-metrics.json`; referência constante calculada pela proporção da classe `low`.

O conjunto é fortemente desbalanceado. Uma decisão constante pela classe `low` alcançaria $10\,694/10\,800 \approx 99{,}02\%$, ligeiramente acima da acurácia do modelo. Portanto, a acurácia geral elevada não demonstra vantagem preditiva nesse teste. O recall nulo de `moderate` impede afirmar que o classificador reconhece adequadamente os níveis intermediários.

Há também uma diferença de cálculo entre os indicadores. O F1 macro reportado inclui os quatro rótulos, atribuindo zero a resultados indefinidos pela convenção `zero_division=0`. A acurácia balanceada corresponde à média dos recalls das classes presentes no teste. Não se

Tabela 6 – Representatividade das classes no teste e sensibilidade observada

Rótulo	Suporte	Recall	Interpretação
`low`	10 694	99,9719%	Classe amplamente predominante.
`moderate`	106	0%	Nenhum exemplo dessa classe foi reconhecido corretamente.
`high`	0	Não estimável	Ausente no conjunto de teste.
`critical`	0	Não estimável	Ausente no conjunto de teste.

Fonte: relatório de classificação do modelo compacto.

devem interpretar esses percentuais como avaliação representativa das quatro condições. Para `high` e `critical`, suporte zero significa falta de evidência, não demonstração de erro ou acerto nessas condições.

6.2 Verificação de software e da exportação

Na revisão do relatório, foram executados 87 testes de backend e IA, sete testes do frontend e o build TypeScript/Vite. As verificações abrangem validação do contrato, compatibilidade com firmware anterior, falhas independentes dos sensores, origem do diagnóstico, persistência CSV e atualização do histórico. A existência de testes aprovados não constitui validação metrológica ou biológica.

O script de verificação compilou o preditor C++ em ambiente nativo e o comparou com o modelo Python em 43 200 registros. Não houve divergência de classe, e a maior diferença de probabilidade foi de aproximadamente $3{,}01 \times 10^{-8}$. Trata-se de evidência de fidelidade da exportação. Essa comparação inclui dados de treinamento e, por isso, não é outra estimativa de generalização do modelo.

O registro de compilação da Heltec V2 descrito na Seção 4 é uma evidência distinta da comparação nativa. Não foram realizados, nesta revisão documental, gravação de firmware, ensaio de calibração, medição de consumo ou cronometragem de inferência na placa. Essas tarefas permanecem no plano experimental.

6.3 Plano de testes físicos e de integração

As Tabelas 7 e 8 definem procedimentos e critérios a serem registrados, sem apresentá-los como resultados já alcançados. Instrumentos de referência devem ter sua identificação e condição de calibração documentadas. Antes dos ensaios, devem ser registrados espécie e massa dos grãos, geometria do mini-silo, posição dos sensores e condições ambientais; esses elementos não estão caracterizados no material técnico examinado.

Tabela 7 – Plano de verificação dos componentes e das falhas

Objeto	Procedimento e instrumento	Evidência a registrar
Alimentação	Medir entrada USB e barramento de 3,3 V com multímetro; observar quedas durante transmissão com osciloscópio, se disponível.	Tensão mínima/máxima, corrente média e picos observáveis; confrontar com a fonte e o regulador efetivos. Não inferir picos apenas da média.
DHT22	Comparar temperatura e UR com termo-higrômetro de referência em condições estáveis e após mudança controlada de ambiente.	Diferenças entre instrumentos, dispersão, tempo de estabilização e ocorrência de leituras inválidas. Não confundir concordância pontual com calibração.
Sonda e MAX6675	Observar repouso, aquecer levemente a ponta com a mão e aguardar estabilização; comparar com termômetro de referência em pontos adequados à montagem. Verificar termopar aberto.	Resposta térmica, diferença em relação à referência e sinalização `probe_ok=false`. Um valor numérico isolado não comprova exatidão.
SW-420	Aplicar perturbações reproduzíveis na estrutura; observar DO e transições com analisador lógico, quando disponível.	Polaridade, limiar ajustado, janela real e repetibilidade. Não converter contagem em intensidade, RPM ou estado de ventilação.
Falhas e IA	Introduzir falhas independentes de DHT22 e termopar, preservando conexões seguras; conferir o payload e a serial.	Medições inválidas null, classe `unavailable`, score/confiança null e continuidade da telemetria válida de vibração.

Fonte: elaboração própria; plano de trabalho, sem resultado físico presumido.

Tabela 8 – Plano de verificação da integração e da validade do classificador

Objeto	Procedimento e instrumento	Evidência a registrar
Execução local	Com sensores válidos, observar serial e interromper o Wi-Fi; registrar a duração da inferência e memória livre em execução.	Classificação local sem depender da API; distribuição de tempos de `inference_us`, reinicializações e comportamento do heap.
Rede e API	Interromper/restabelecer Wi-Fi e servidor; enviar payloads válidos e inválidos em ambiente de teste isolado.	Retornos 201/422/503 quando aplicáveis, estados de frescor, reconexão e inexistência de classificação reapresentada como nova.
CSV e interface	Reiniciar a API, consultar o histórico, exportar o CSV e interromper a rede do navegador durante uma coleta de teste.	Persistência das linhas aceitas, indicadores de validade, origem `esp32`, preservação de dados antigos e atualização de novos pontos.
Validade do classificador	Definir protocolo de coleta por lotes independentes, com referências físicas e, quando cabível, análise microbiológica por laboratório habilitado.	Rótulos externos às previsões, matriz de confusão, recall por classe, incerteza das métricas e avaliação de lotes não usados no ajuste.

Fonte: elaboração própria; plano de trabalho, sem resultado físico presumido.

Para investigar fungos, os rótulos futuros não devem ser produzidos apenas pela própria temperatura ou pela classe prevista pelo modelo: isso criaria uma avaliação circular. A aquisição ambiental e a determinação de referência devem ser independentes. O planejamento também deve considerar variações de tipo de grão, teor de água, localização da sonda e fontes externas de aquecimento.

7 Conclusões sobre a viabilidade da arquitetura

A revisão do MS-Monitor identifica uma arquitetura coerente para aquisição de medições, execução local de um classificador compacto, comunicação por HTTP e persistência dos registros. O código separa a telemetria de vibração dos atributos térmicos e higrométricos, trata falhas dos sensores e permite inferência local independente do envio pela rede. A compilação registrada e a comparação Python–C++ sustentam a viabilidade computacional preliminar.

Essa conclusão é limitada à implementação e às verificações descritas. Não demonstra desempenho de detecção de fungos, contaminação ou micotoxinas. O desbalanceamento do teste, a falta de exemplos de duas classes e o recall nulo da classe moderada impedem atribuir validade agronômica ao classificador atual. Seu uso apropriado, nesta etapa, é a demonstração de aquisição, inferência embarcada e integração de dados em uma atividade acadêmica.

A continuidade do projeto exige caracterização física da montagem, comparação

metrológica dos sensores, medição do consumo e avaliação do tempo de inferência na ESP32. A investigação da relação com condições de deterioração dependerá de dados experimentais representativos e referências independentes. Controle e classificação de ventilação permanecem fora da implementação analisada.

Declaração de apoio de inteligência artificial

Ferramentas de IA generativa foram utilizadas como apoio à revisão textual e técnica, à organização do documento LaTeX e à elaboração de diagramas. As afirmações foram confrontadas com o código e as fontes indicadas, e exemplos simulados foram identificados como tais. Esse apoio não substitui a revisão dos integrantes, a avaliação docente ou os ensaios experimentais. A responsabilidade pela versão submetida permanece com os autores.

Referências

Adafruit Industries. *DHT22 temperature-humidity sensor: technical details*. [S.l.], s.d. Disponível em: <https://www.adafruit.com/product/385>. Acesso em: 25 out. 2026.

Analog Devices. *MAX6675: Cold-Junction-Compensated K-Thermocouple-to-Digital Converter*. [S.l.], 2021. Revisão 3, jun. 2021. Disponível em: <https://www.analog.com/media/en/technical-documentation/data-sheets/MAX6675.pdf>. Acesso em: 25 out. 2026.

BREIMAN, L. Random forests. *Machine Learning*, v. 45, p. 5–32, 2001. Disponível em: <https://www.stat.berkeley.edu/~breiman/randomforest2001.pdf>. Acesso em: 25 out. 2026.

Espressif Systems. *ESP32 Series: datasheet*. [S.l.], s.d. Versão 5.3. Disponível em: <https://documentation.espressif.com/esp32_datasheet_en.pdf>. Acesso em: 25 out. 2026.

FREITAS, C. A. O. d. *Trabalho de IA Embarcada: especificação dos projetos*. 2026. Enunciado fornecido à turma. Grupo G3, p. 2–3. Universidade Federal do Amazonas.

Grupo 03. *MS-Monitor: código-fonte e documentação técnica*. 2026. Revisão 07b1400; firmware, API, frontend e relatório do modelo embedded-rf-6b0d1696547d. Material técnico do projeto.

Heltec Automation. *WiFi LoRa 32(V2): specifications*. [S.l.], s.d. Disponível em: <https://heltec.org/project/wifi-lora-32v2/>. Acesso em: 25 out. 2026.

PEDREGOSA, F. et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011. Disponível em: <https://jmlr.org/papers/v12/pedregosa11a.html>. Acesso em: 25 out. 2026.

World Health Organization. *Mycotoxins*. 2023. Disponível em: <https://www.who.int/news-room/fact-sheets/detail/mycotoxins>. Acesso em: 25 out. 2026.